

ANÁLISIS Y DISEÑO ORIENTADO A OBJETO

ANÁLISIS Y DISEÑO DE SISTEMA INF-3290

Introducción

Modelos de diseño con la metodología UML

1

Modelos

CONCEPTOS ORIENTADOS A OBJETOS

- La programación orientada a objetos Vs programación por procedimiento. La POO examina los objetos que forman parte de un sistema. Cada objeto es una representación de alguna cosa o evento real.
- Objetos Los objetos son personas, lugares o cosas relevantes para el sistema a analizar. Algunos objetos comunes son clientes, artículos, pedidos, etcétera. Los objetos también pueden ser pantallas de GUI o áreas de texto en la pantalla.

CONCEPTOS ORIENTADOS A OBJETOS

- Clases. Por lo general, los objetos forman parte de un grupo de elementos similares, conocidos como clases.
- Los objetos se representan y agrupan mediante clases, las cuales son óptimas para la reutilización y la facilidad de mantenimiento.
- Una clase define el conjunto de atributos compartidos y comportamientos que se encuentran en cada objeto de la clase.

MODELOS A TRABAJAR

- 1.Software Requirement Specification (SRS).
- 2. Casos de uso.
- 3. Escenarios.
- 4. Diagrama de secuencia.
- 5.Diagrama de clase.
- 6. Diagrama de actividad.
- 7.Diseño de la interfaz gráfica de usuario (GUI).

A CONSIDERAR

El modelado de casos de uso es la base de todas las técnicas de UML. Se usa para derivar las actividades, secuencias y clases.

DOS TIPOS DE MODELOS

MODELOS Estáticos (organizativos)	**MODELOS Dinámicos (Funcionales, de comportamiento).**
SRS	Escenarios
Caso de uso	Diagrama de secuencia
Diagrama de clase	Diagrama de actividad
	Diseño de la Interfaz Gráfica de Usuario

HERRAMIENTAS PARA CREAR LOS MODELOS

Para crear los modelos o diagramas usamos herramientas CASE (Computer-Aided Software Engineering), por ejemplo:

1. Visio de Microsoft
2. Umlet
3. Draw.io
4. Visual paradigm online

Esta segunda esta disponible de forma gratuita en la siguiente dirección www.unlet.com

A CONSIDERAR

- Lo que se pasa por alto con frecuencia en el desarrollo de un nuevo sistema es que **entre más progrese un proyecto**, **más costosas serán las modificaciones** en los requerimientos de negocios de un sistema.
- Es mucho más fácil, rápido y menos costoso modificar el diseño de un sistema mediante el uso de una herramienta CASE, o incluso en papel, durante las fases de análisis y diseño de un proyecto, que hacerlo durante la fase de desarrollo.

LENGUAJE UNIFICADO DE MODELADO

UML es una metodología de diseño que consiste en una serie de modelos textuales y gráficos de la solución propuesta. Estos modelos definen el alcance del sistema.

UML fue creada desde principios de 1980. Es una forma estándar y sistemática para modelar el diseño de programas POO.

Esta industria fue creada y manejada por la Object Management Group (OMG).



LENGUAJE UNIFICADO DE MODELADO

- Definen la interacción del usuario con el sistema y como los componentes del sistema interactúan entre si para implementar su funcionalidad.
- La fase de diseño es una de la más importante en el proceso o ciclo de desarrollo de software.
- Un proyecto de software fallido puede estar asociado a un pobre diseño y a una comunicación inadecuada entre desarrollador y stakeholders (partes interesadas).



MODELO SRS

Especificación de Requerimiento de Software (Software Requirement Specification –SRS-).

2

SRS

MODELO SRS
ESPECIFICACIÓN DE REQUERIMIENTO DE SOFTWARE (SOFTWARE REQUIREMENT SPECIFICATION –SRS-).

PROPÓSITOS

- Definir los requerimientos funcionales del sistema.
- Identificar los limites del sistema.
- Identificar usuarios.
- Describir la interación entre el sistema y los usuarios externo.
- Establecer comunicación entre clientes y desarrolladores para describir el sistema deseado.
- Dar paso al siguiente modelo (caso de uso) ofreciendo los elementos básicos.

SRS

- Para producir este modelo se debe hacer una entrevista al dueño del negocio, empresa u organización y a los usuarios finales del sistema.
- El objetivo de la entrevista es documentar los procesos del negocio y establecer el alcance del sistema.
- El resultado es un documento formal que detalla los requerimientos del sistema. Este documento establece los acuerdos entre las partes interesadas y el desarrollador.

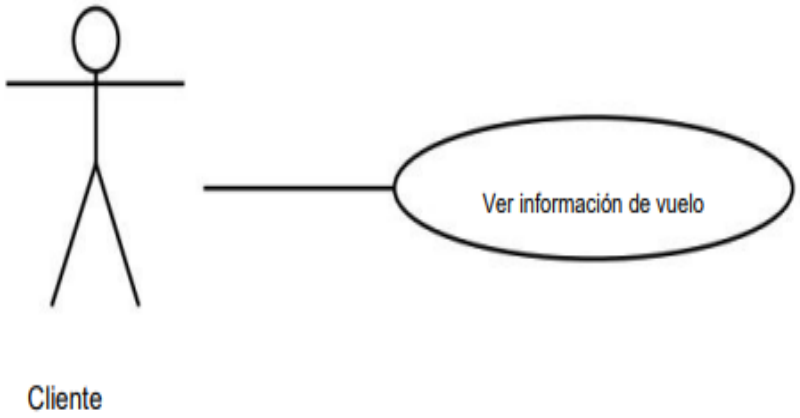
SRS

- **El SRS no hace referencia a los requerimientos técnicos del sistema.**
- Una vez desarrollado este modelo (SRS) los requerimientos funcionales contenido en el mismo se transforman en un diagrama de caso de uso (segundo modelo en el proceso de diseño). Previo al diagrma se identifican los casos de uso, sus actores, descripción, post y pre condición, entre otros.

3

MODELO

CASO DE USO



MODELO CASO DE USO

- Modelan los servicios que proporcionará el sistema.
- Este modelo describe como las **entidades externas** pueden usar un sistema. Las entidades externas pueden ser: humanos, recurso de hardware u otros sistemas. En el contexto de UML estas entidades son conocidos como **actores.**
- El caso de uso ayuda a definir el alcance y limite del sistema. Suele hacerse en forma de diagrama con una descripción de la interacción que se produce.

MODELO CASO DE USO

Los casos de uso son desarrollados tomando como punto de partida el modelo SRS. **Cada interacción que ocurre entre un actor y el sistema debe ser modelado como un caso de uso**.

El actor es cualquier entidad externa que interactúa con el sistema, puede ser:

- Un usuario (humano) por ejemplo, un cliente.
- Otro sistema, por ejemplo, un software de facturación
- Un recurso de hardware, por ejemplo, un sensor de temperatura.

EJEMPLO TIPOS DE ACTORES

Humano / Usuario Humano

- **Cliente:** utiliza un sistema de comercio electrónico para comprar
- **Administrador del Sistema:** gestiona la configuración, usuarios y seguridad del sistema.
- **Empleado de Ventas:** registrar clientes potenciales, gestionar oportunidades y realizar ventas.
- **Paciente:** interactúa con un sistema de historia clínica para ver su información o programar citas.
- **Conductor:** obtener indicaciones y planificar rutas en aplicación de navegación GPS

EJEMPLO TIPOS DE ACTORES

Sistema de Software

- **Sistema de Pago en Línea:** Un sistema externo, ej. PayPal que interactúa con un sistema de comercio electrónico
- **Sistema de Autenticación Centralizado** proporciona servicios de autenticación y autorización a otras aplicaciones.
- **Servicio de Correo Electrónico (SMTP Server):** Un sistema que el sistema principal utiliza para enviar notificaciones o confirmaciones por correo
- **Sistema de Gestión de Inventario:** Un sistema externo que proporciona información sobre los niveles de stock de productos al sistema de ventas.

EJEMPLO TIPOS DE ACTORES

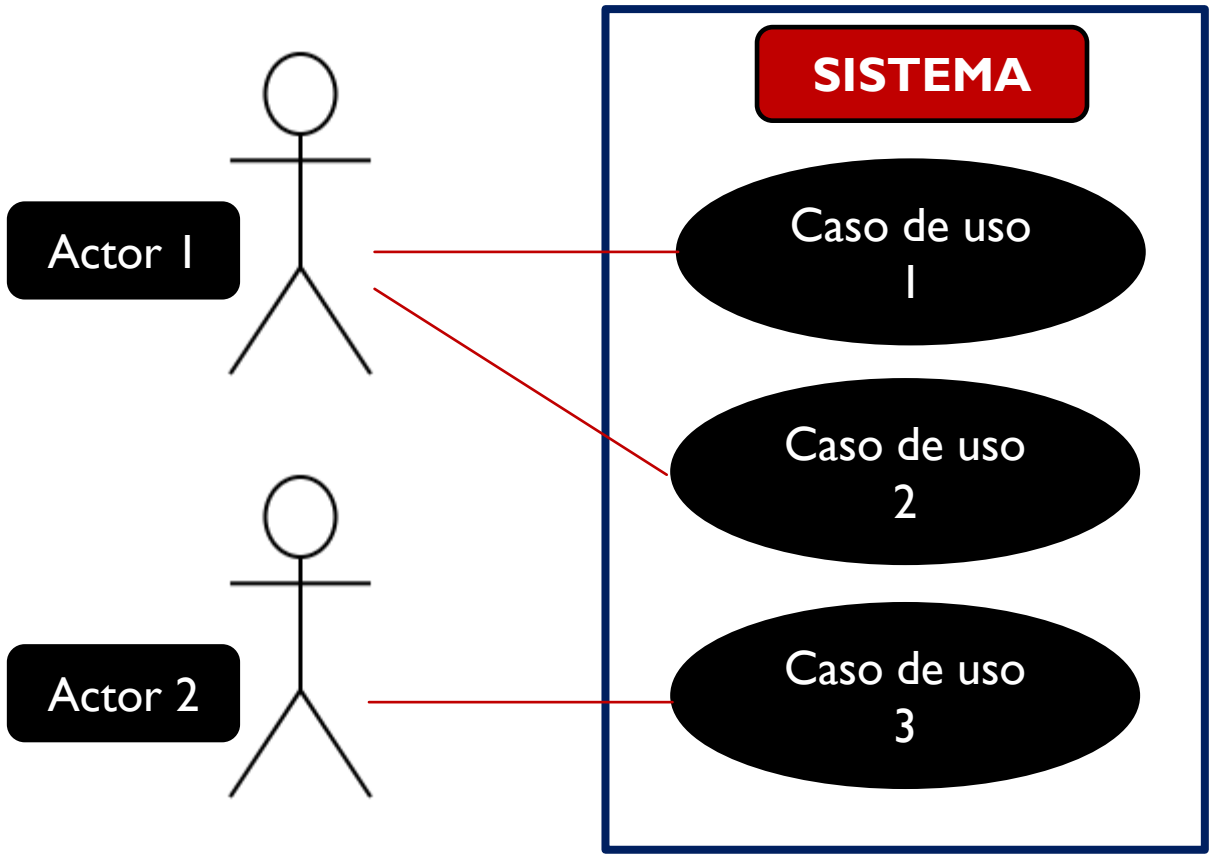
Dispositivo de Interfaz

- **Cajero Automático (ATM):** permite a los usuarios interactuar con un sistema bancario.
- **Sensor de Temperatura:** Envía lecturas de temperatura a un sistema de monitoreo ambiental.
- **Lector de Código de Barras:** Utilizado en un punto de venta para escanear códigos de barras y obtener información de productos de un sistema de inventario.

CASO DE USO

- Recuerde que los casos de uso no se centran en cómo el sistema realizará las funciones, sino en **qué funciones deben incorporarse** en el sistema desde el punto de vista del usuario.
- Un error común al desarrollar casos de uso es incluir acciones iniciadas por el propio sistema. **El énfasis del caso de uso está en la interacción entre las entidades externas y el sistema.**

MODELO CASO DE USO



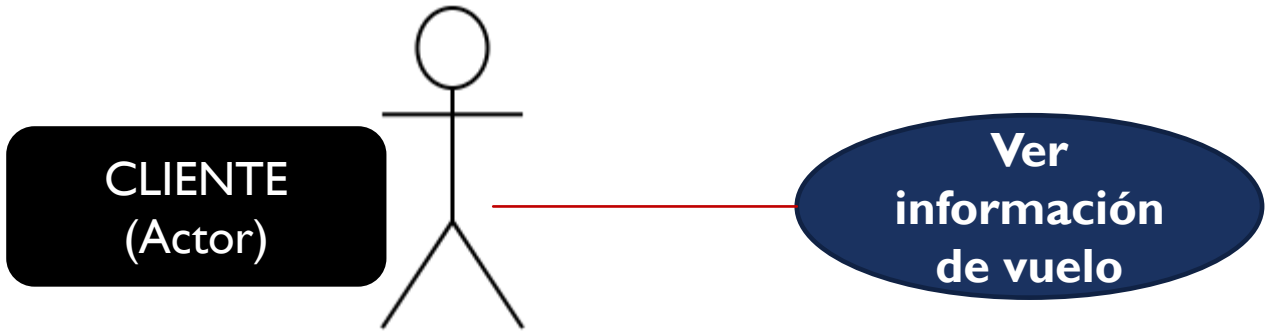
EJEMPLO.
DIAGRAMA DE CASO DE USO (diagrama genérico). Con dos actores y tres casos de uso.

Nota: Estos casos de uso se convierten en procesos del sistema.

MODELO CASO DE USO

Con este modelo se representa cada interacción que ocurre entre un actor y el sistema. Por ejemplo, para la creación del sistema de una aerolínea se puede considerar el siguiente caso de uso: **ver información de vuelo.**

Este a su vez es uno de los requerimientos del modelo SRS.



MODELO CASO DE USO

Se debe evitar los siguientes errores en el modelo de casos de uso:

1. **No debe incluirse acción inicializada por el sistema.**

Ejemplo: El sistema envía una confirmación al cliente vía correo electrónico.

2. No debe incluir una descripción de los requerimientos técnicos del sistema.

Es importante recordar que el enfoque no es cómo el sistema desarrolla las funciones, sino cuáles funciones deben ser incorporadas en el sistema desde el punto de vista del usuario.

PROCESOS ALTERNATIVOS

PROCESOS ALTERNATIVOS

Existen procesos alternativos que se convierten en Casos de Uso y otros que NO.

No todo camino alternativo o excepción en un sistema amerita ser elevado a la categoría de un nuevo caso de uso dentro del diagrama.

PROCESOS ALTERNATIVOS:

1) Casos de uso alternativos.

2) Flujos alternativos. No se considera un caso de uso.

Situación 1:
Procesos alternativos que son Casos de Uso

Generalmente modelados con relaciones <<extend>> o de forma independiente

- Representa un **subproceso complejo** con su propia secuencia de pasos (múltiples interacciones entre el actor y el sistema).

- Cuando un proceso alternativo requiere una secuencia compleja de intercambio de mensajes, protocolos o llamadas a servicios con un actor que no es una persona, y cumple con tener un objetivo propio, se modela como un caso de uso independiente (usualmente mediante <<extend>> o <<include>>).

EL ACTOR. El concepto de actor va más allá del ser humano. El actor puede ser un sistema informático externo, un dispositivo o un ser humano.

Ejemplo #1

Procesos alternativos con actores no humanos que se convierten en casos de uso. El actor es un sistema externo.

Actor: Sistema Externo o infraestructura de pago externa (Pasarela de Pagos)

Caso Base: Realizar Compra en Línea (Interacción inicial con el cliente humano).

Proceso alternativo / Caso de Uso extendido: Procesar pago a través de una plataforma externa o pasarela de pago. (*Stripe, PayPal, Authorize.net, Paycomet*, etc.).

Durante el flujo de pago, el sistema principal establece una interacción con la infraestructura de pago externa. Esa secuencia de múltiples mensajes constituye un caso de uso independiente.

Ejemplo #2

Procesos alternativos con actores no humanos que se convierten en casos de uso. El actor es un sistema externo.

Actor: Proveedor de Identidad Externo (Servidor OAuth / Auth0)

Caso Base: Iniciar Sesión en la Aplicación (El usuario intenta entrar).

Proceso alternativo / Caso de Uso extendido: Autenticar mediante Proveedor Federado o servicio externo como Google, Auth0.

Si el usuario opta por no usar el formulario interno y presiona "Iniciar sesión con Google", el sistema redirige al usuario pero, a nivel técnico, establece una interacción máquina a máquina (intercambio de tokens JWT, redirección de callbacks, consulta de perfiles) con los servidores de Google o Auth0. Este subproceso alternativo de autenticación federada es un caso de uso propio donde el actor externo es el *Servidor de Google* u otra plataforma.

Ejemplo #3

Procesos alternativos con actores no humanos que se convierten en casos de uso. El actor es un dispositivo o hardware.

Sistema de Control de Acceso a Estacionamiento.

Actor: Cámara de Reconocimiento de Placas (Dispositivo de Hardware ANPR - Automatic Number Plate Recognition).

Caso Base: Registrar ingreso con tarjeta RFID ()

Proceso alternativo / Caso de Uso extendido: Autorizar ingreso por Reconocimiento Automático de Placa

Secuencia de Pasos del Caso de Uso Extendido (Interacción con el Hardware)

1. El conductor detiene el vehículo frente a la estación de acceso.
2. El sensor de presencia de piso detecta el vehículo y envía una alerta al sistema principal de estacionamiento.
3. El sistema principal envía una orden digital (comando de captura) a la Cámara ANPR (dispositivo de hardware).
4. La Cámara ANPR activa su obturador infrarrojo, toma una fotografía de alta resolución de la placa del vehículo.
5. La Cámara ANPR transmite de vuelta al sistema principal una cadena de texto estructurada con el número de la placa.
6. El sistema principal consulta la base de datos de vehículos registrados y constata que la placa cuenta con autorización.
7. Al cumplirse la validación positiva, el sistema principal envía la orden eléctrica al mecanismo de la barrera física para permitir el paso del vehículo.

Ejemplo #4

Procesos alternativos con ACTORES HUMANOS que se convierten en casos de uso.

Sistema de Gestión de Compras en Línea (E-commerce).

Actor: El Cliente (que asume temporalmente el rol fiscal/empresarial durante la compra).

Caso Base: Realizar compra de producto.

Proceso alternativo / Caso de Uso extendido: Solicitar Facturación a Nombre de una Persona Jurídica (Empresa)

Considerar otro ejemplo, RECUPERAR CONTRASEÑA

Secuencia de pasos del caso de uso extendido

1. Durante el paso de confirmación del pedido, el cliente (actor humano) marca la casilla opcional: *"Requiero factura con validez fiscal para empresa"*.
2. El sistema principal detecta la selección y activa el caso de uso extendido, mostrando un formulario de datos fiscales en pantalla.
3. El cliente ingresa manualmente el Número de Registro Fiscal (RNC/NIT) de la empresa.
4. El sistema valida el formato del número ingresado contra las reglas preestablecidas.
5. El cliente introduce la Razón Social o nombre legal de la empresa y selecciona el régimen fiscal correspondiente en un menú desplegable.
6. El cliente revisa los datos empresariales introducidos y presiona el botón "Guardar y Asociar a la Factura".
7. El sistema valida que los datos estén completos y actualiza el pedido para que, al finalizar el cobro, la pasarela de facturación genere un Comprobante Fiscal Especial para empresas, retornando el control al flujo principal de cierre.

Situación 2:
Procesos alternativos que no son casos de uso

Flujos alternativos. No se considera un caso de uso.

Un proceso alternativo no se convierte en un caso de uso cuando:

- Es una reacción local y simple del sistema ante un error (ej. mostrar un mensaje de error y reintentar).
- No aporta un valor funcional independiente ni constituye un objetivo de negocio autónomo.

Ejemplo 1:
Procesos alternativos que no son casos de uso

Flujos alternativos. No se considera un caso de uso.

Caso Base: Registrar Nuevo Estudiante

Proceso alternativo que NO es Caso de Uso: *Campo Obligatorio Vacío* o *Formato de Correo Inválido*.

Explicación: Si el profesor/administrador deja un campo en blanco, el sistema detecta el error, resalta el campo en rojo y muestra un mensaje de advertencia ("El correo electrónico no es válido"). Esto se describe textualmente en el paso 3a del caso de uso base (*El sistema notifica el error y regresa al paso 2*), pero jamás se dibuja como un óvalo de caso de uso en el diagrama.

Ejemplo 2:
Procesos alternativos que no son casos de uso

- **Caso Base:** Retirar Dinero en Cajero Automático
- **Proceso alternativo que NO es Caso de Uso:** *Fondos Insuficientes en la Cuenta.*
- *Explicación:* Cuando el cliente solicita una cantidad superior a su saldo, el cajero despliega una pantalla informativa indicando el rechazo de la transacción y ofrece regresar al menú principal. Es una excepción del flujo normal que se resuelve de manera inmediata dentro del mismo alcance del caso de uso de retiro.

Ejemplo 3:
Procesos alternativos que no son casos de uso

- **Caso Base:** Subir Tarea en Plataforma Moodle
- **Proceso alternativo que NO es Caso de Uso:** *Archivo con Formato No Permitido* o *Exceso de Peso*.
- *Explicación:* Si el estudiante intenta adjuntar un archivo ejecutable (.exe) o un archivo que supera los 50 MB, el sistema emite un aviso de rechazo. Este comportamiento alternativo no amerita un diagrama ni un caso de uso propio; es simplemente una regla de validación descrita en el flujo de excepción de la especificación del caso de uso.

Donde representar los flujos alternativos

- El Diagrama de Secuencia (Sequence Diagram)

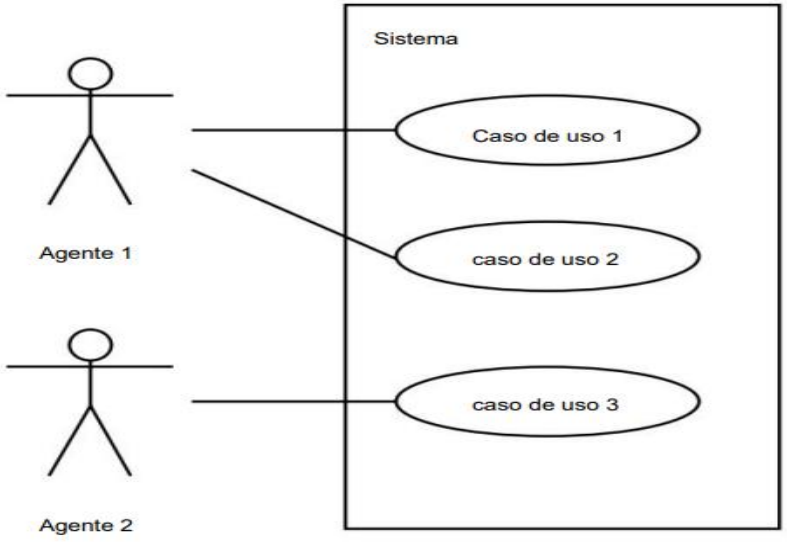
Este diagrama modela la interacción de los objetos y actores a lo largo del tiempo mediante el intercambio de mensajes.

- El Diagrama de Actividad (Activity Diagram)

Es el heredero natural del flujograma de procesos. Es ideal para modelar la lógica de negocio detallada de un caso de uso, incluyendo todos sus caminos alternativos y de excepción.

4

Diagrama de
CASO DE USO



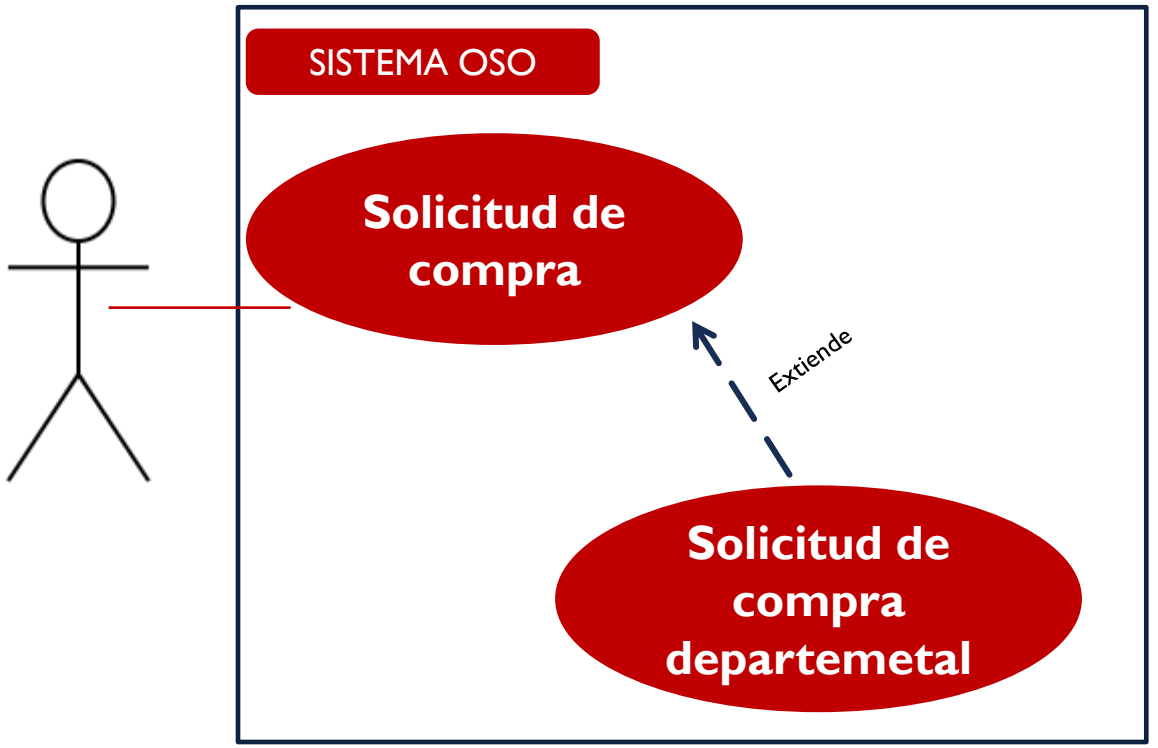
RELACIONES ENTRE CASOS DE USO

Según el paso #4 de la guía:

“La segunda acción consiste en buscar cualquier relación que pueda existir entre los casos de uso. Relaciones de inclusión y la relación de extensión. Ver relación de inclusión y extensión de los casos de uso de la aplicación OSO en la página 47”

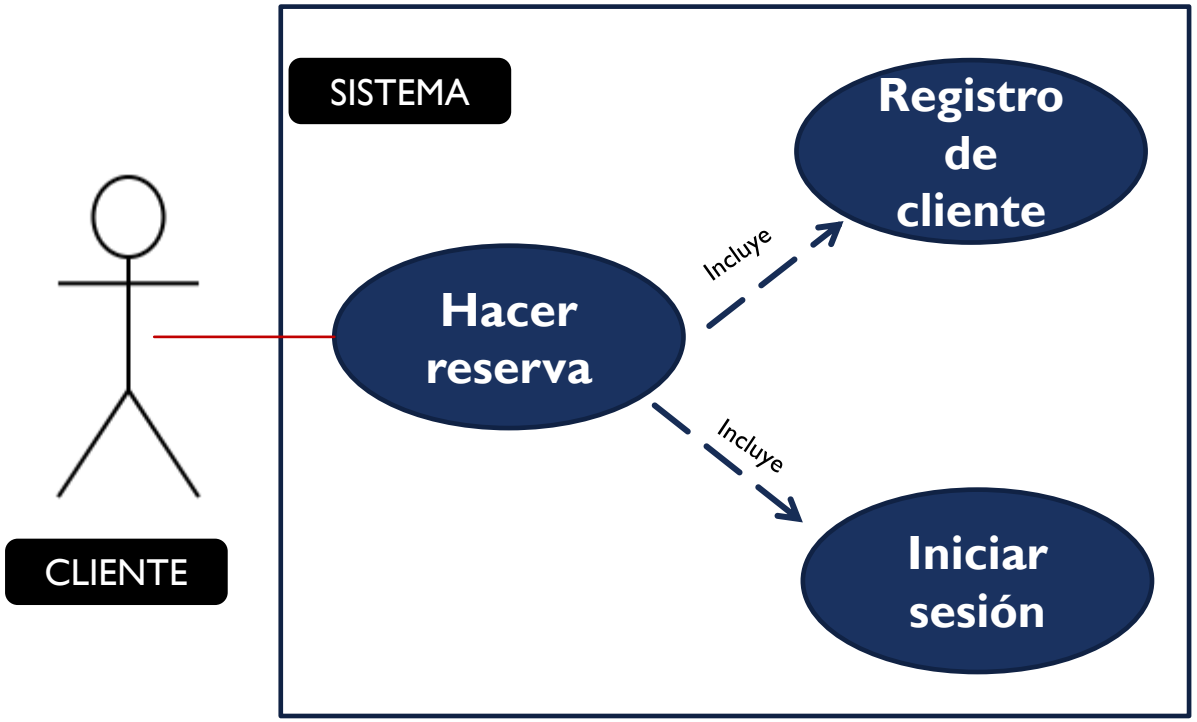
RELACIÓN DE EXTENSIÓN ENTRE CASOS DE USO

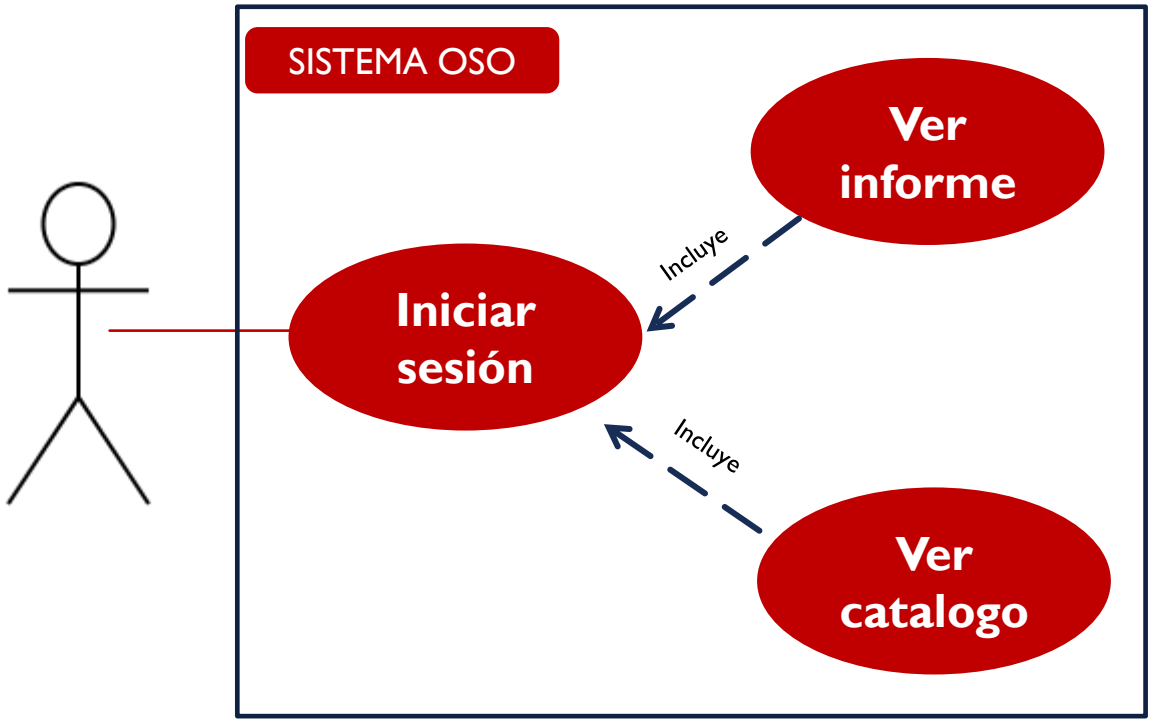
Esta relación ocurre cuando un caso de uso principal es la base para otros casos de uso. El caso de uso base se amplía con otros casos de uso que se derivan de el.



RELACIÓN DE INCLUSION ENTRE CASOS DE USO

Esta relación ocurre cuando un caso de uso incluye otro caso de uso, donde el caso de uso que se incluye debe ejecutarse como condición previa.





No puede usar el caso de uso Hacer reserva independientemente del caso de uso registro de cliente

RELACIÓN DE INCLUSIÓN Y EXTENSIÓN EN CASOS DE USO

- Cuando un caso de uso incluye otro caso de uso, el caso de uso que se incluye debe ejecutarse como condición previa. Pero puede usar el caso de uso que se incluye independientemente del caso de uso principal.

ESCENARIOS

A partir de los Casos de Uso

5

1. Al usuario se le presenta un cuadro de diálogo de inicio de sesión.
2. El usuario ingresa un nombre de inicio de sesión y una contraseña.
3. El usuario envía la información.
4. El nombre y la contraseña se comprueban y verifican.
5. Al usuario se le presenta una pantalla de solicitud de suministro.

ESCENARIO

El escenario describe como se ejecutara el caso de uso. Su propósito es describir los pasos que deben realizar internamente los objetos que componen el sistema.

ESCENARIO

Consiste en profundizar en las descripciones de los casos de uso realizadas en el paso 3 (3.2) de la guía y crear un escenario detallado, indicando como se llevará a cabo el caso de uso.

ESCENARIO

Ejemplo tomado del sistema OSO

Escenario para el caso de uso **ver catálogo de suministros**

1) El usuario ha iniciado sesión y se le ha asignado un nivel de estado de usuario.
2) A los usuarios se les presenta una tabla de catálogo que contiene una lista de suministros. La tabla contiene información como el nombre del suministro, la categoría, la descripción y el costo.
3) Los usuarios pueden filtrar los suministros por categoría
4) Los usuarios tienen la opción de cerrar sesión o realizar una solicitud de compra.

EJEMPLO DE ESCENARIO SISTEMA OSO

El siguiente **ESCENARIO** describe una secuencia posible de acciones para llevar a cabo el caso de uso **INICIAR SESION.**

- **Aquí veremos que un caso de uso puede tener varios escenarios. Escenario #1 representa el flujo principal, Escenario #2 representa el flujo alternativo.**

Escenario #1 caso de uso inicio de sesion

1. Al usuario se le presenta un cuadro de diálogo de inicio de sesión.
2. El usuario ingresa un nombre de inicio de sesión y una contraseña.
3. El usuario envía la información.
4. El nombre y la contraseña se comprueban y verifican.
5. Al usuario se le presenta una pantalla de solicitud de suministro.

EJEMPLO DE ESCENARIO SISTEMA OSO

El siguiente **ESCENARIO** describe una secuencia posible de acciones para llevar a cabo el caso de uso **INICIAR SESION.** Veste segundo escenario es el flujo alternativo, cuando el sistema no se comporta conforme a lo esperado.

Escenario #2 caso de uso inicio de sesion

1. Al usuario se le presenta un cuadro de diálogo de inicio de sesión.
2. El usuario ingresa un nombre de inicio de sesión y una contraseña.
3. El usuario envía la información.
4. **El nombre y la contraseña se comprueban pero no se pueden verificar.**
5. Se informa al usuario de la información de inicio de sesión incorrecta.
6. Al usuario se le presenta nuevamente un cuadro de diálogo de inicio de sesión.
7. El usuario vuelve a intentarlo o cancela la solicitud de inicio de sesión.

ESCENARIO

A considerar:

La descripción del caso de uso **reserva de vuelo** describe la relación del usuario con el sistema al momento de reservar un vuelo.

Descripción del caso de uso: El cliente suministra el número de vuelo e indica el asiento solicitado. Después de hacer la solicitud recibe la información de confirmación. El cliente entro al sistema con su usuario, tiene información sobre el vuelo desde la pantalla de reserva. El cliente recibe un e-mail de confirmación con los detalles del vuelo y las políticas de cancelación de vuelo.

EJERCICO GRUPAL #1

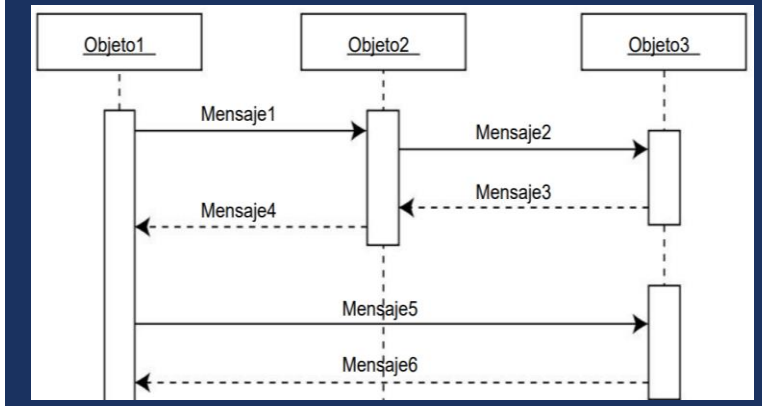
Crear un escenario para el caso de uso **reserva de vuelo.**



Descripción: El cliente suministra el número de vuelo e indica el asiento solicitado. Después de hacer la solicitud recibe la información de confirmación. El cliente entro al sistema con su usuario, tiene información sobre el vuelo desde la pantalla de reserva. El cliente recibe un e-mail de confirmación con los detalles del vuelo y las políticas de cancelación de vuelo.

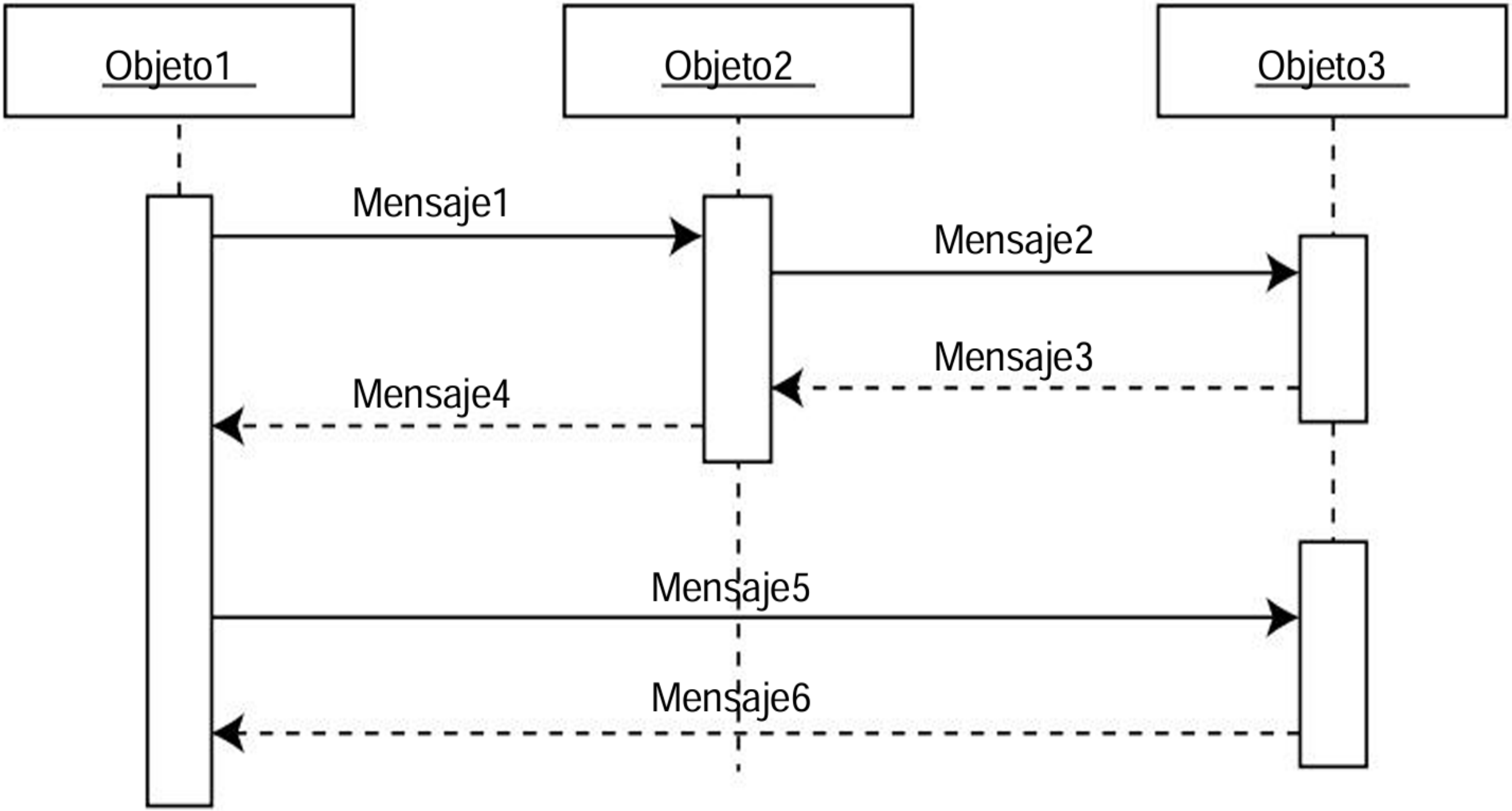
Diagramas de
SECUENCIA

6



LOS DIAGRAMAS DE INTERACCIÓN

- Los diagramas de interacción se dividen en diagrama de secuencia y diagrama de colaboración.
- **DIAGRAMA DE SECUENCIA.** Determinar los objetos que estarán involucrados en cada escenario, la operación que realiza el objeto, cómo interactúan los objetos, **se basa en un escenario**, es decir, se hace a partir del escenario.
- Los mensajes representados en el diagrama de secuencia formarán la base de los métodos de las clases del sistema.
- A los diagramas de secuencia en ocasiones se los conoce como diagramas de eventos o escenarios de eventos.


Objeto1
Objeto2
Objeto3
Mensaje1
Mensaje2
Mensaje3
Mensaje4
Mensaje5
Mensaje6

EJEMPLO ANTERIOR

ESCENARIO

Ejemplo tomado del sistema OSO

Escenario para el caso de uso **ver catálogo de suministros**

1) El usuario ha iniciado sesión y se le ha asignado un nivel de estado de usuario.
2) A los usuarios se les presenta una tabla de catálogo que contiene una lista de suministros. La tabla contiene información como el nombre del suministro, la categoría, la descripción y el costo.
3) Los usuarios pueden filtrar los suministros por categoría
4) Los usuarios tienen la opción de cerrar sesión o realizar una solicitud de compra.

- A partir de este escenario, puede ver que necesita un método de la clase **catalogo de producto** que devolverá una lista de categorías de productos.
- Otro método que necesita la clase es uno que devuelva una lista de productos filtrada por categoría. El diagrama de secuencia muestra la interacción que se produce entre la clase Comprador (UI) y la clase Catálogo de Producto.

DIAGRAMA DE SECUENCIA PARA EL ESCENARIO VER CATALOGO DE SUMINISTRO

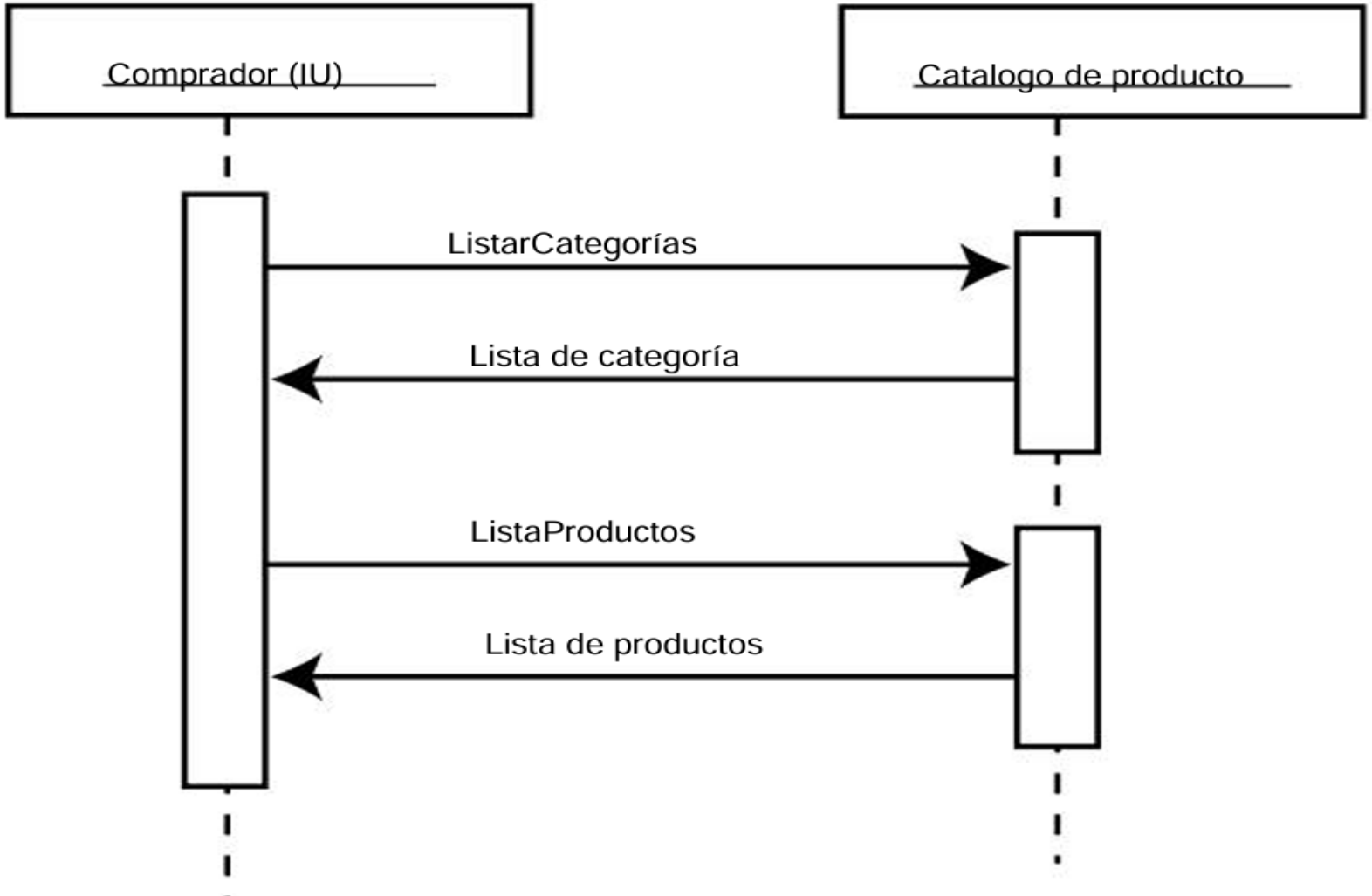
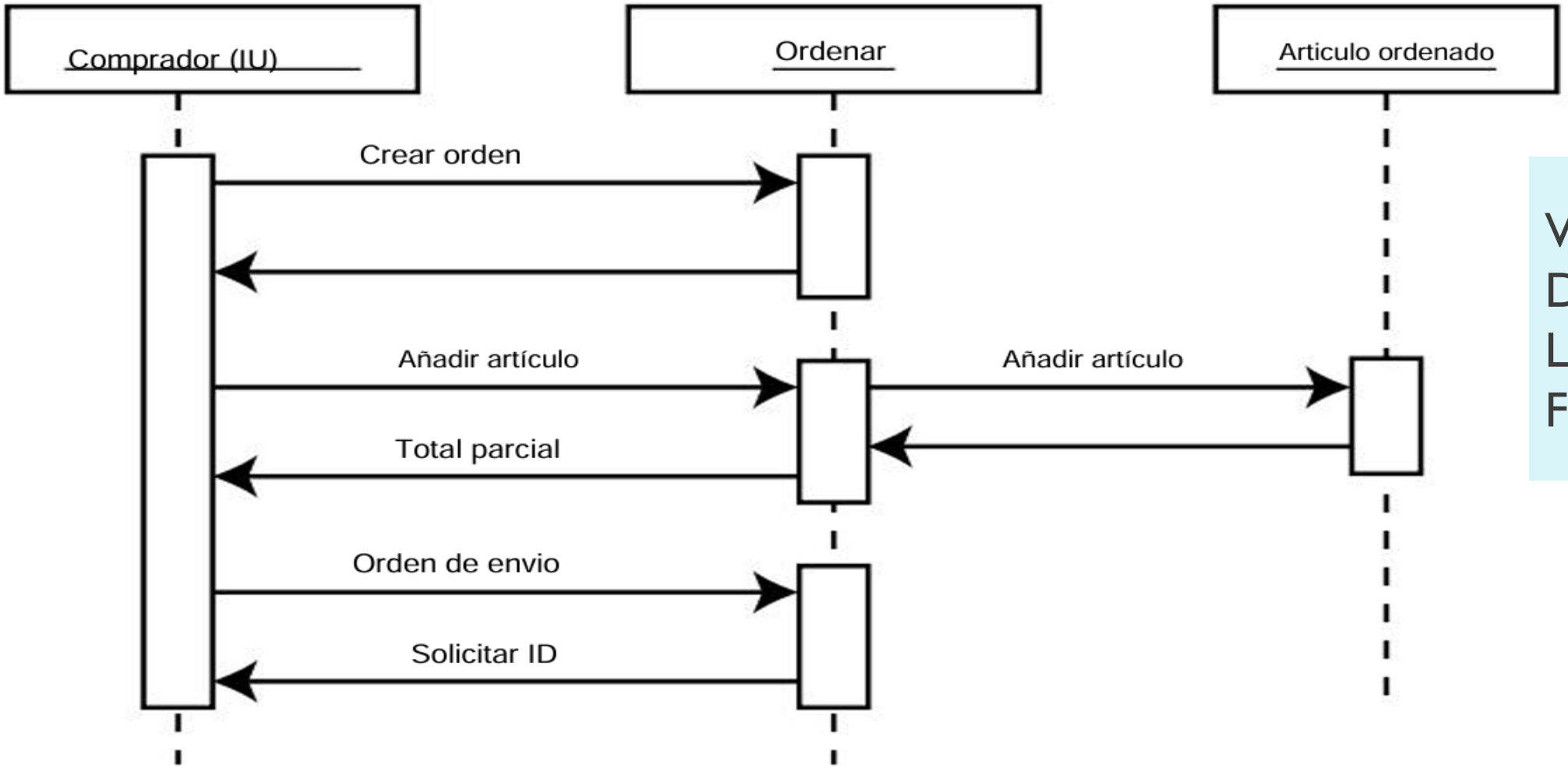


DIAGRAMA DE SECUENCIA PARA EL ESCENARIO SOLICITUD DE COMPRA



VER DETALLES EN LA PAG. 54 FIGURA 4-13

DIAGRAMA DE SECUENCIA PELICULA EN STOCK CASO DE USO ALQUILER DE PELICULA

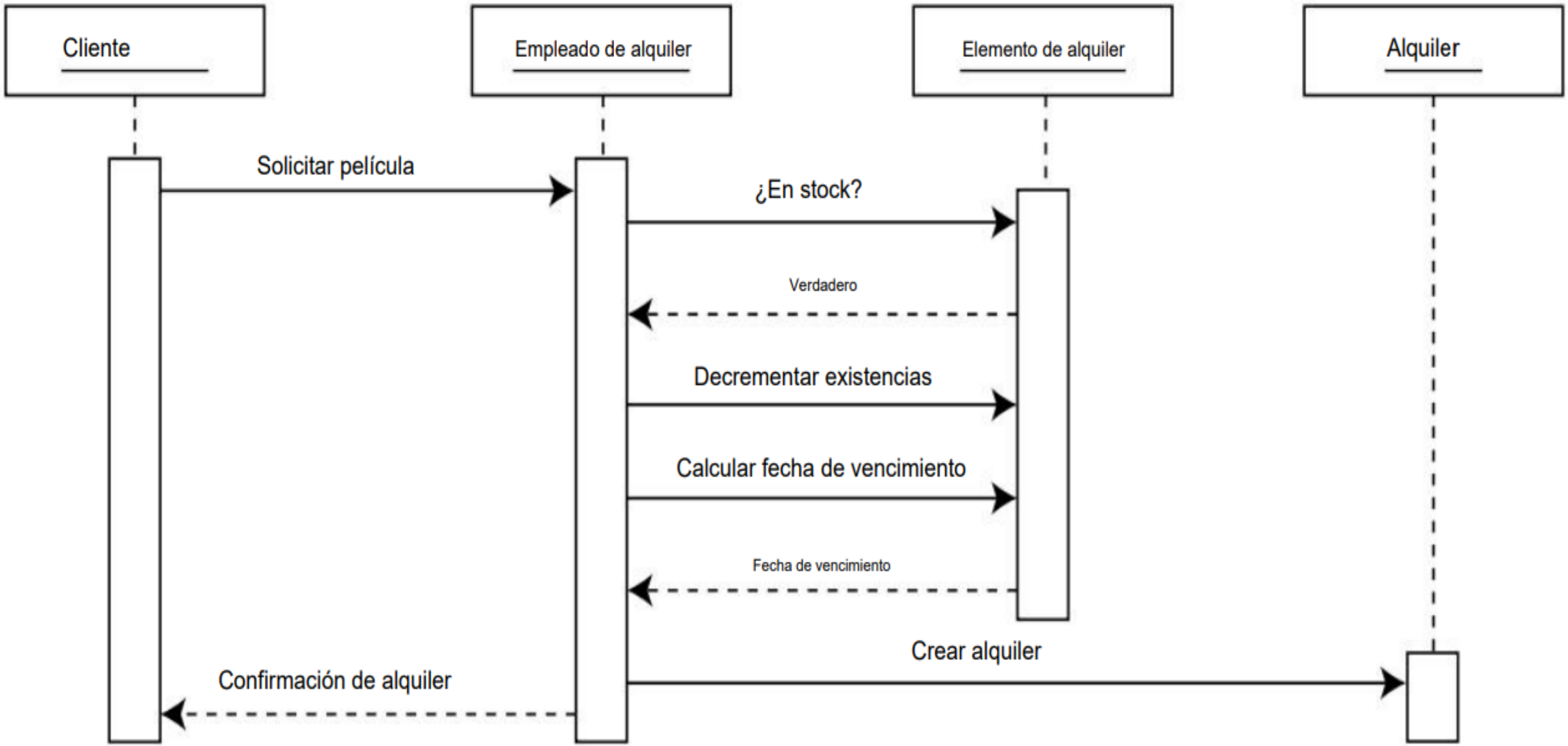


DIAGRAMA DE SECUENCIA

- A medida que analiza el diagrama de secuencia, obtiene una comprensión de las clases de objetos que estarán involucrados en el procesamiento del programa; también comprenderá qué métodos necesitará para crear y adjuntar a esas clases. También debe modelar las clases y los métodos representados en el diagrama de secuencia en el diagrama de clases.
- **Estos documentos de diseño deben tener referencias cruzadas y revisarse continuamente según sea necesario.**

CLASES Y ATRIBUTOS

7

IDENTIFICAR LAS CLASES Y SUS ATRIBUTOS

Se puede implementar varias estrategias para identificar las clases:

- Profundizar en cada caso de uso y definir una serie de pasos necesarios para llevarlo a cabo. **Se trata de identificar las clases a partir del escenario**.
- Identificar elementos gramaticales (nombres comunes) en las descripciones de casos de uso y el SRS, los cuales podrían ser candidatos a posibles clases.

IDENTIFICAR LAS CLASES Y SUS ATRIBUTOS

A) Profundizar en cada caso de uso y definir una serie de pasos necesarios para llevarlo a cabo. **Se trata de identificar las clases a partir del escenario**.

EJEMPLO

A partir del **Escenario** para el caso de uso **ver catálogo de suministro** se puede identificar una clase que será responsable de recuperar la información de la base de datos y filtrado de los productos.

CLASE: Catalogo de Productos.

NOTA: Revisar el ejemplo en el tema sobre escenarios (diapositivas anteriores) para notar que en el escenario #2 del caso de uso iniciar sesión hay algunos atributos.

IDENTIFICAR LAS CLASES Y SUS ATRIBUTOS

Ejemplo Sistema OSO

NO.	CASOS DE USO	POSIBLES CLASES
1	Iniciar sesión	Usuario, nombre de usuario, contraseña, éxito, fracaso
2	Ver catálogo de suministro	Usuario, tabla de catálogo, lista de suministros, información, nombre del suministro, categoría, descripción, costo
3	Solicitud de compra	Comprador, artículos, carrito, número, artículo solicitado, costo, costo total
4	Solicitud de compra departamental	Gerente de departamento, artículos, carrito, número, artículo solicitado, costo, costo total, solicitud de compra del departamento

IDENTIFICAR LAS CLASES Y SUS ATRIBUTOS

B) Identificar elementos gramaticales (nombres comunes) en las descripciones de casos de uso y el SRS, los cuales podrían ser candidatos a posibles clases.

Elementos del modelo	Elemento grammatical relacionado	Ejemplo
Clases	**NOMBRES COMUNES** Haciendo una lista de nombres comunes, luego se procede a identificar cuales aplican para formar las clases.	Usuario Vuelo Avion
Métodos	**VERBOS** Buscando y aislando los *verbos* del texto se puede encontrar las operaciones	Reservar Consultar Confirmar
Atributos	**ADJETIVOS** Característica, cualidad, propiedad, particularidad, peculiaridad, singularidad, rasgo, marca, carácter, idiosincrasia, naturaleza o condición del objeto;	Corporativo Nombre Correo

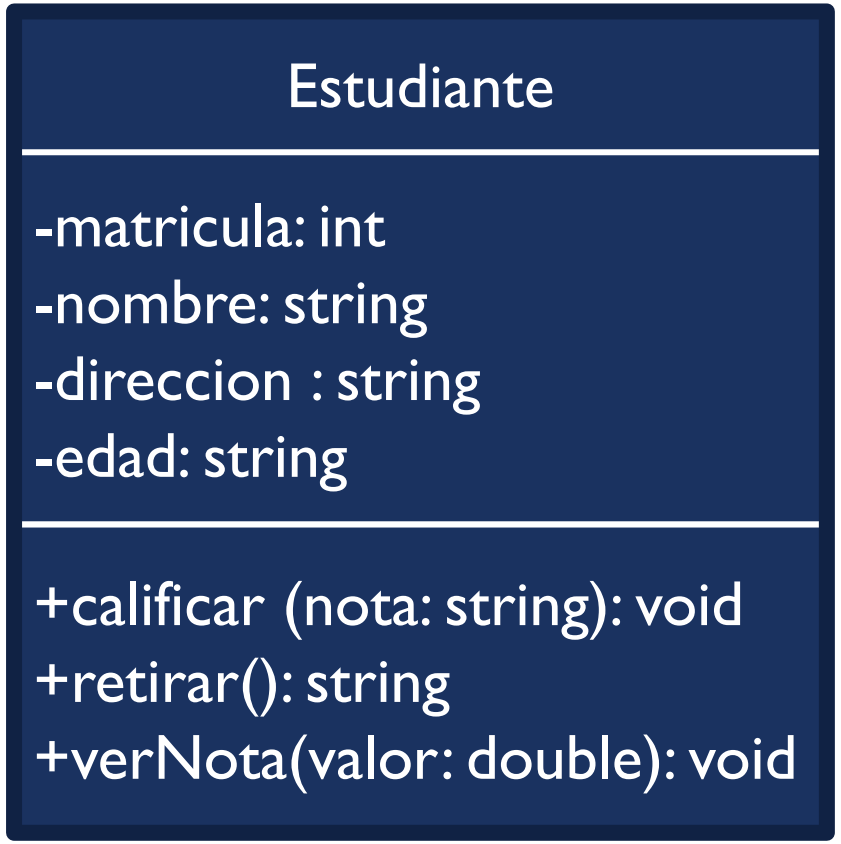
IDENTIFICAR LAS CLASES Y SUS ATRIBUTOS

Al definir la estructura de la clase se determina a través de sus atributos (campos) y operaciones (métodos) el tipo de información que manejara.

A partir de los escenarios se puede identificar procesos (metodos) y vincularlos a las clases elegidas. **Los mismos se veran representados en los diagramas de secuencia.**

IDENTIFICAR LAS CLASES Y SUS ATRIBUTOS

Notación UML de una clase



Paso #7 de la guia

No.	Clase	Atributos	Métodos	Tipo de datos
I	Estudiante	Matricula Nombre Dirección Edad	Calificar Retirar verNota	Entero Cadena Cadena …
.	.	.		.
.	.	.		.
n	n	n		n

SINTAXIS DE MIEMBROS DE UNA CLASE

La sintaxis para escribir los miembros de la clase es como sigue:

- **Atributos o campos:** es recomendable que el acceso siempre sea: **-**

acceso (- , +) ID : tipo = valor inicial { información adicional: permite agregar información sobre el comportamiento o las precondiciones/postcondiciones. }

Ejemplo: - salarioMensual : double = 93456.23 { DOP }

- **Métodos u operaciones:** es recomendables que el acceso siempre sea **+**

acceso (- , + , #) ID (par1: tipo, par2: tipo, ...) : tipo de retorno { info. Adicional:}

Ejemplo: + pagar(monto : double) : boolean { si es true fue OK el pago }

SINTAXIS DE MIEMBROS DE UNA CLASE

A CONSIDERAR

Bajo el enfoque de la metodología UML (Lenguaje Unificado de Modelado), durante el proceso de análisis y diseño, se crean varios modelos que sirven como insumos cruciales para la posterior creación de la base de datos. Estos modelos ayudan a comprender los requisitos del sistema desde diferentes perspectivas y a refinar la estructura de los datos que se almacenarán.

¿Cómo se utilizan estos modelos como insumos para la base de datos?

Clases del Diagrama de Clases se convertirán en Tablas: Las clases identificadas en el diagrama de clases a menudo se convierten en tablas en la base de datos.

Atributos del Diagrama de Clases se convertirán en Columnas: Los atributos de las clases se transforman en las columnas de las tablas. El tipo de dato especificado en el diagrama de clases (o inferido) guía la elección del tipo de dato de la columna en la base de datos.

Relaciones del Diagrama de Clases → Claves Foráneas y Tablas de Cruce: Las asociaciones entre clases (especialmente las de multiplicidad diferente de uno a uno) se implementan mediante claves foráneas que referencian las claves primarias de otras tablas.

el Diagrama de Clases es el modelo UML más directo y fundamental para el diseño de la base de datos. Proporciona la estructura estática de los datos. Los otros diagramas, especialmente los de casos de uso y los diagramas de comportamiento (actividades y secuencia), complementan esta información al revelar los requisitos funcionales y el flujo de datos, asegurando que la base de datos pueda soportar las operaciones del sistema de manera efectiva.

Es importante destacar que el proceso no siempre es una traducción directa uno a uno. A veces, es necesario realizar ajustes y optimizaciones durante el diseño de la base de datos para mejorar el rendimiento, la normalización y la escalabilidad. Sin embargo, los modelos UML proporcionan una base sólida y una comprensión clara de los datos requeridos por el sistema.

EL DISEÑO DE LA BASE DE DATOS ESTA FUERA DEL ALCANCE DE ESTE CURSO. Diseño lógico de datos o modelo entidad-relación

EJERCICO GRUPAL #2

EJERCICO GRUPAL

- Leer el RSR y hacer una tabla con las posibles clases, atributos y operaciones.



EJERCICO GRUPAL #3

Crear un diagrama de secuencia modelando el escenario de reserva de vuelo.



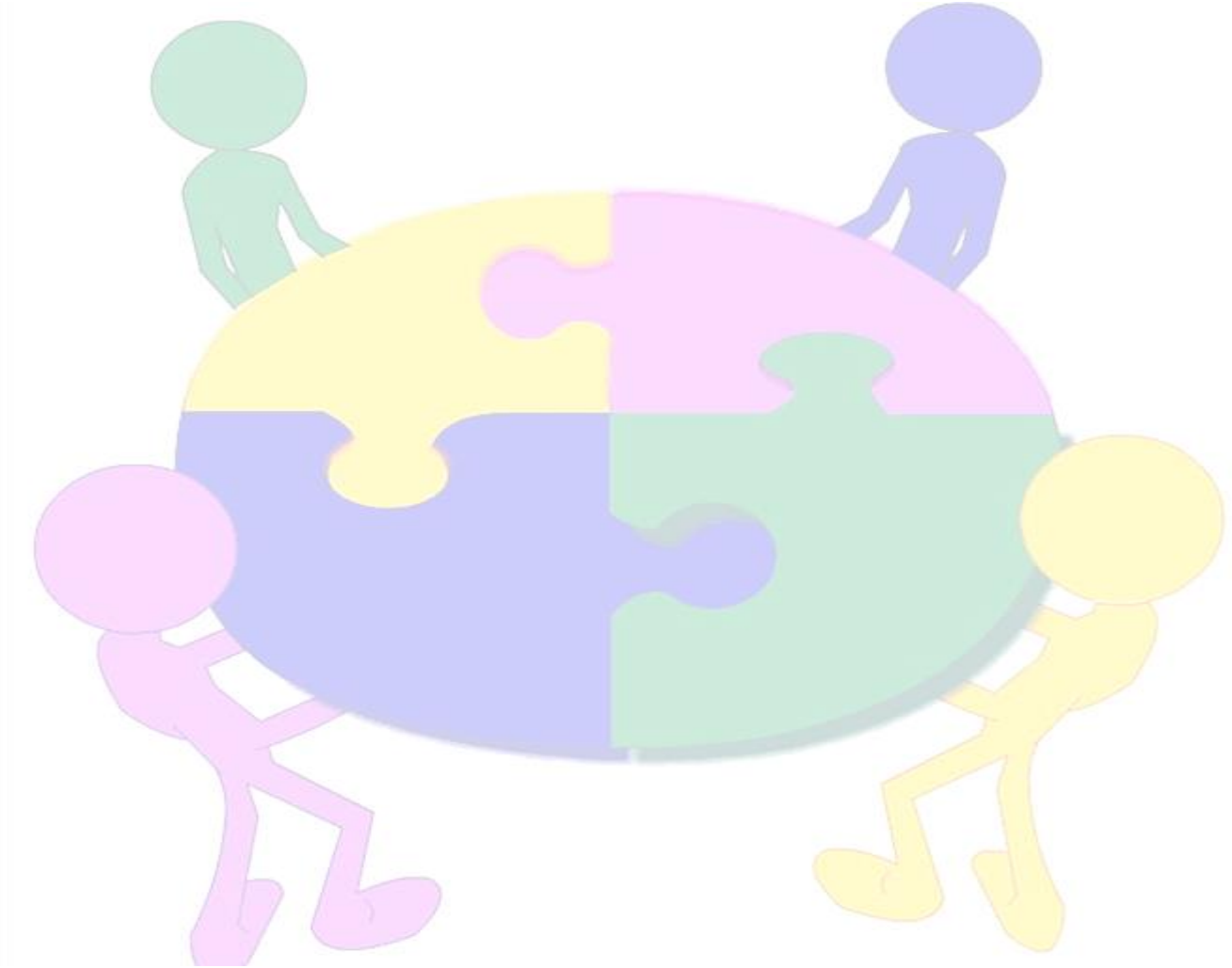


DIAGRAMA DE CLASES

8

AGREGAR ASOCIACION ENTRE CLASE

OPERACIONES DE LAS CLASES / METODOS

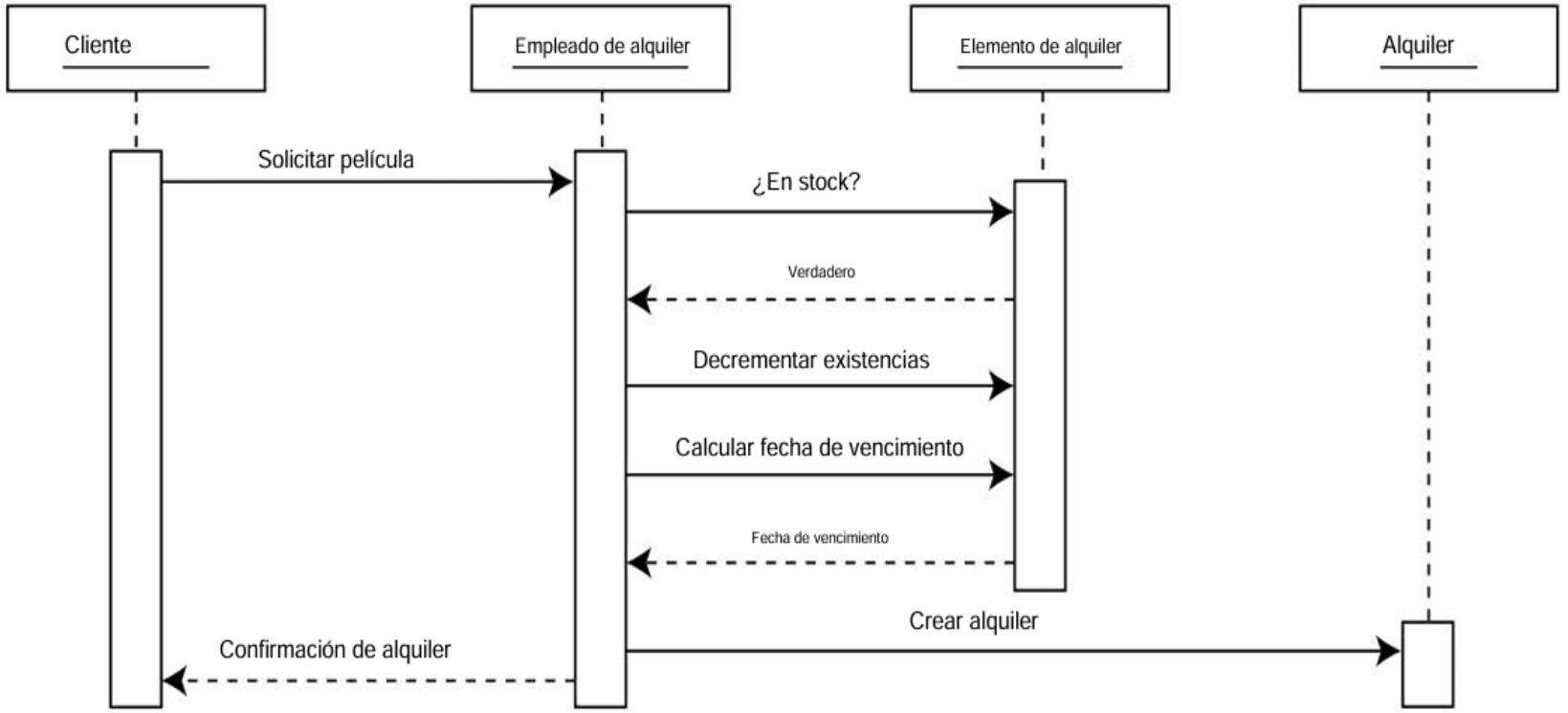
- **Se trata de identificar las clases a partir del escenario**. Con los **diagramas de secuencia** se determina los objetos que estarán involucrados en cada escenario, la operación que realiza el objeto.
- **Los mensajes representados en el diagrama de secuencia** formarán la base de los métodos de las clases del sistema.

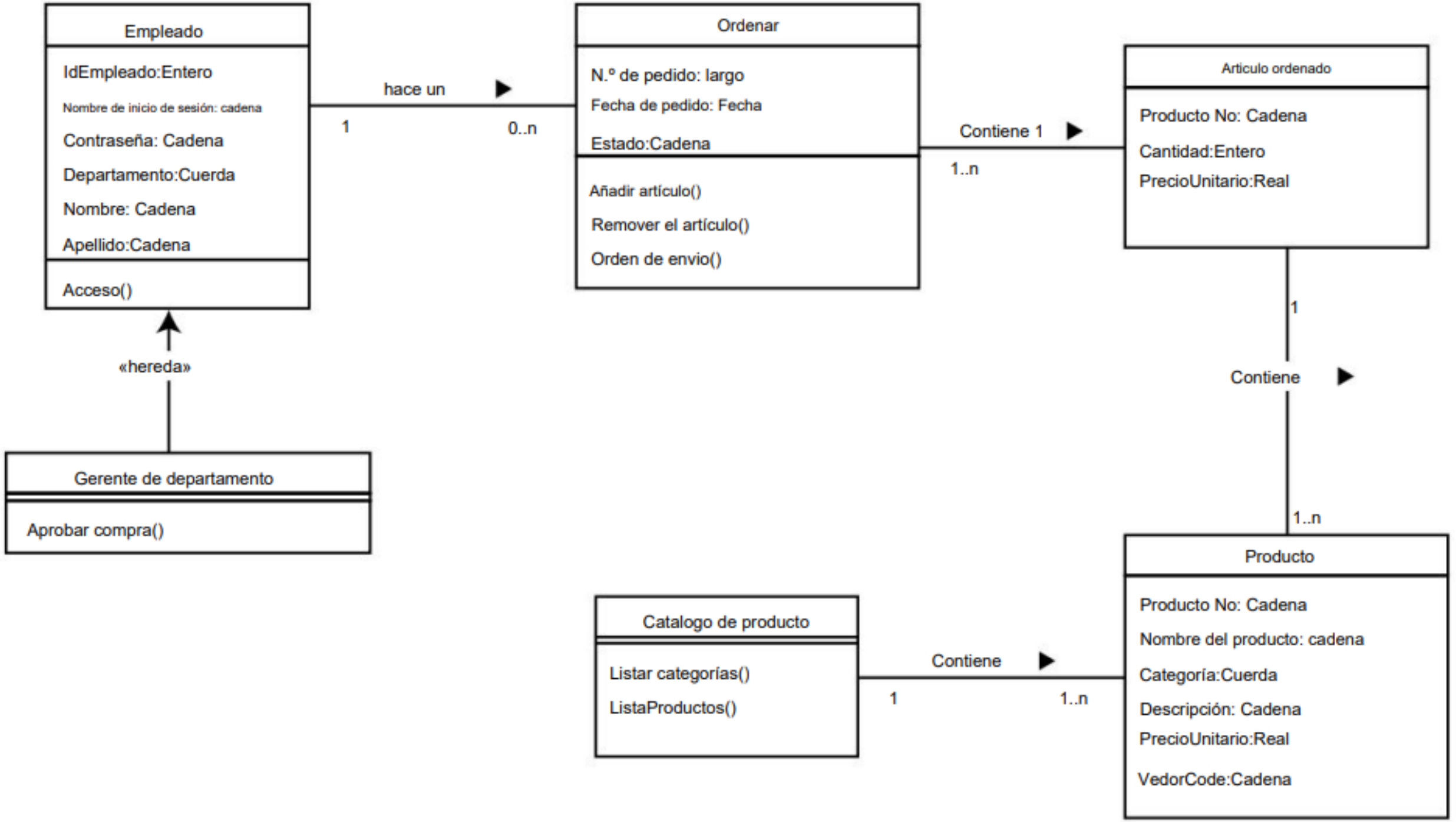
DIAGRAMA DE CLASE

- A medida que analiza el diagrama de secuencia, obtiene una comprensión de los objetos que estarán involucrados en el procesamiento del programa; también comprenderá qué métodos necesitará para crear y adjuntar a las clases.
- **Debes modelar las clases y los métodos representados en el diagrama de secuencia en el diagrama de clases.**

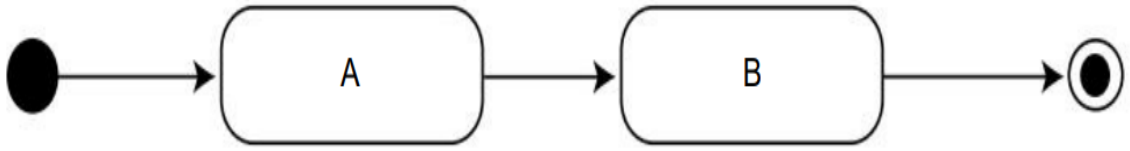
OPERACIONES DE LAS CLASES

Este diagrama de secuencia tiene 4 objetos interactuando, cada uno de los cuales apunta a una clase y cada uno de los mensajes son operaciones o metodos.




Empleado
IdEmpleado:Entero
Nombre de inicio de sesión: cadena
Contraseña: Cadena
Departamento:Cuerda
Nombre: Cadena
Apellido:Cadena
Acceso()
hace un
1
0..n
Ordenar
N.º de pedido: largo
Fecha de pedido: Fecha
Estado:Cadena
Añadir artículo()
Remover el artículo()
Orden de envio()
Contiene 1
1..n
Artículo ordenado
Producto No: Cadena
Cantidad:Entero
PrecioUnitario:Real
1
Contiene
1..n
«hereda»
Gerente de departamento
Aprobar compra()
Catalogo de producto
Listar categorías()
ListaProductos()
Contiene
1
1..n
Producto
Producto No: Cadena
Nombre del producto: cadena
Categoría:Cuerda
Descripción: Cadena
PrecioUnitario:Real
VedorCode:Cadena

Diagramas de ACTIVIDAD



9

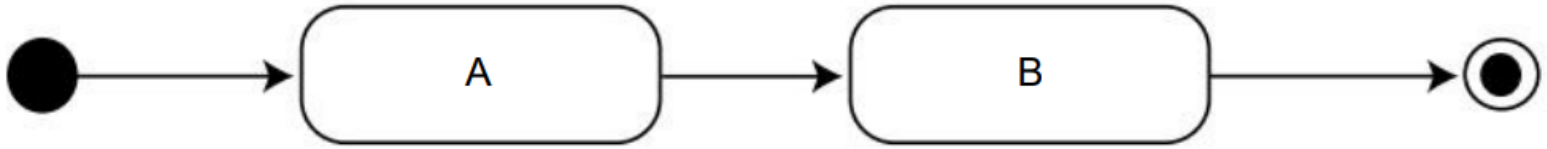
PÁGINA 34

LIBRO BEGINNING C# OBJECT ORIENTED PROGRAMMING, DEL AUTOR DAN CLARK.

DIAGRAMA DE ACTIVIDAD

Es una representación visual del flujo de ejecución de un proceso u operación. Ilustra el **flujo de actividades** que deben ocurrir durante una operación o proceso.

El propósito del diagrama de actividad es modelar el flujo de control de una actividad a otra a medida que el programa procesa.



NIVELES DE ENFOQUES DEL DIAGRAMA DE ACTIVIDAD:

Nivel Alto. A nivel de sistema, representa cada caso de uso como una actividad. ´

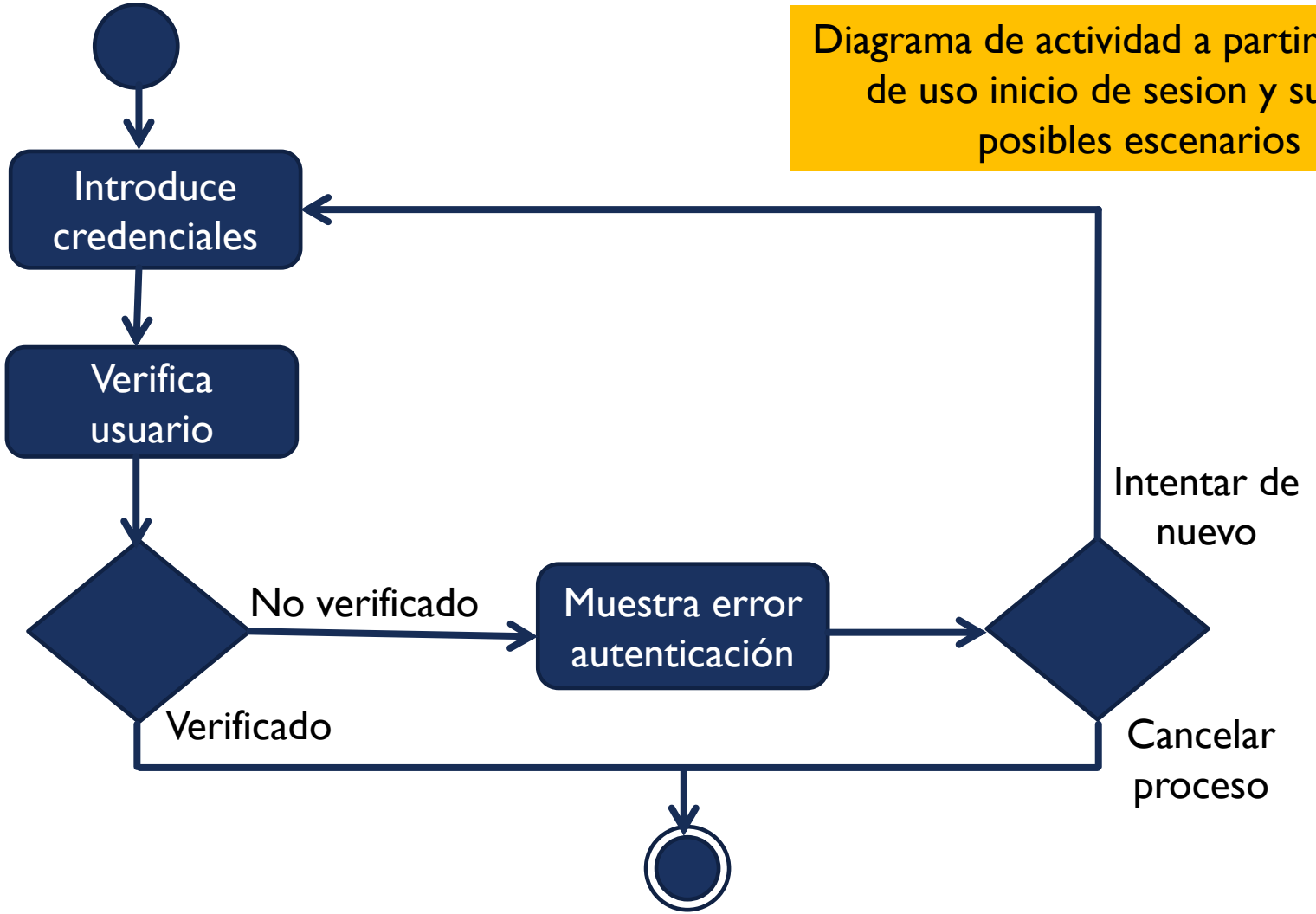
Un alto enfoque a nivel de sistema representa cada caso de uso como una actividad y diagrama el flujo de trabajo entre los diferentes casos de uso.



NIVELES DE ENFOQUES DEL DIAGRAMA DE ACTIVIDAD:

- **Nivel Medio.** Diagrama el flujo de trabajo que ocurre dentro de un caso de uso particular. **(ENFOQUE QUE USAREMOS).**
- **Nivel Bajo**. Diagrama el flujo de trabajo que ocurre dentro de una operación particular de una de las clases del sistema.

Diagrama de actividad a partir del caso de uso inicio de sesion y sus dos posibles escenarios



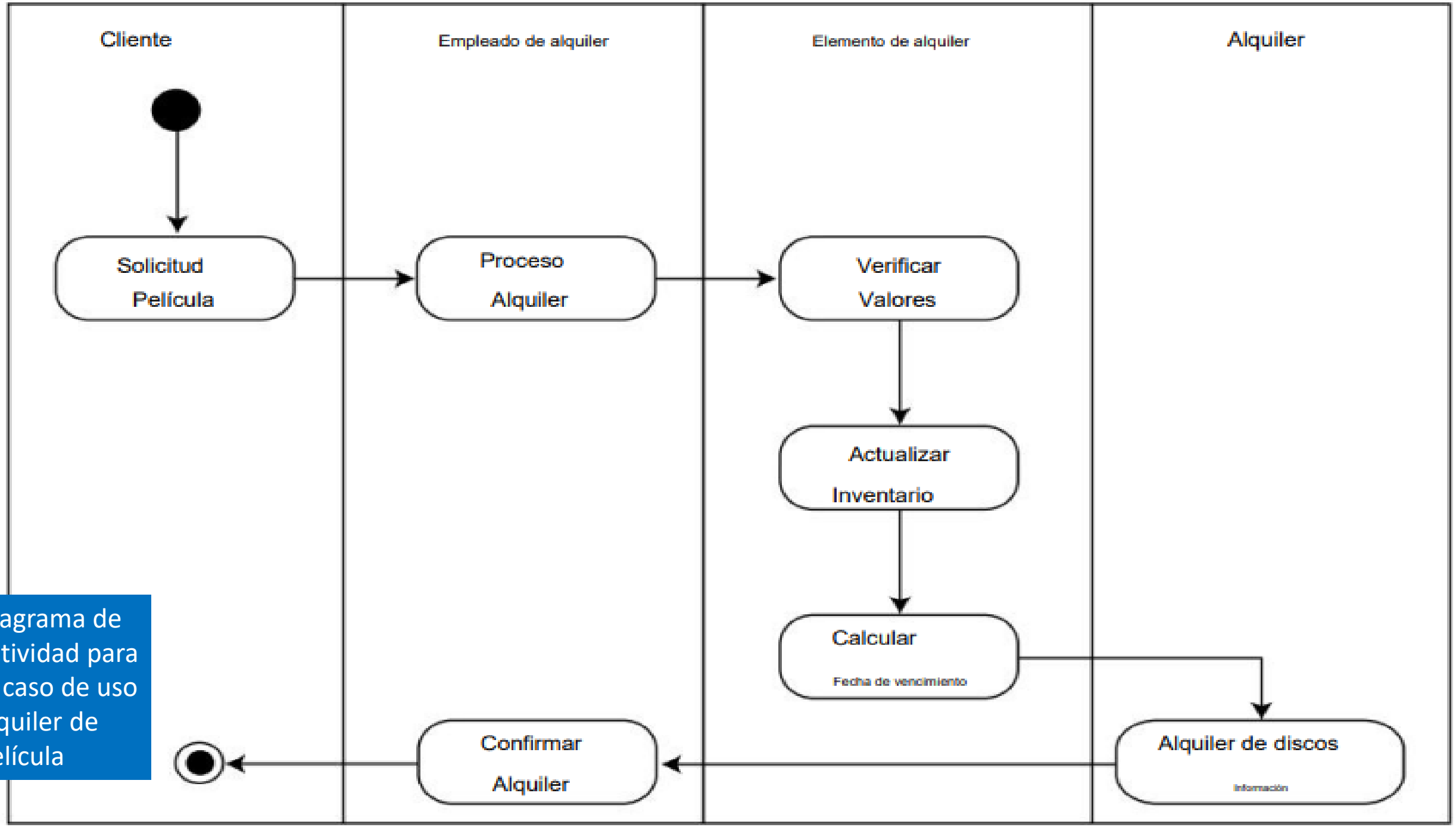
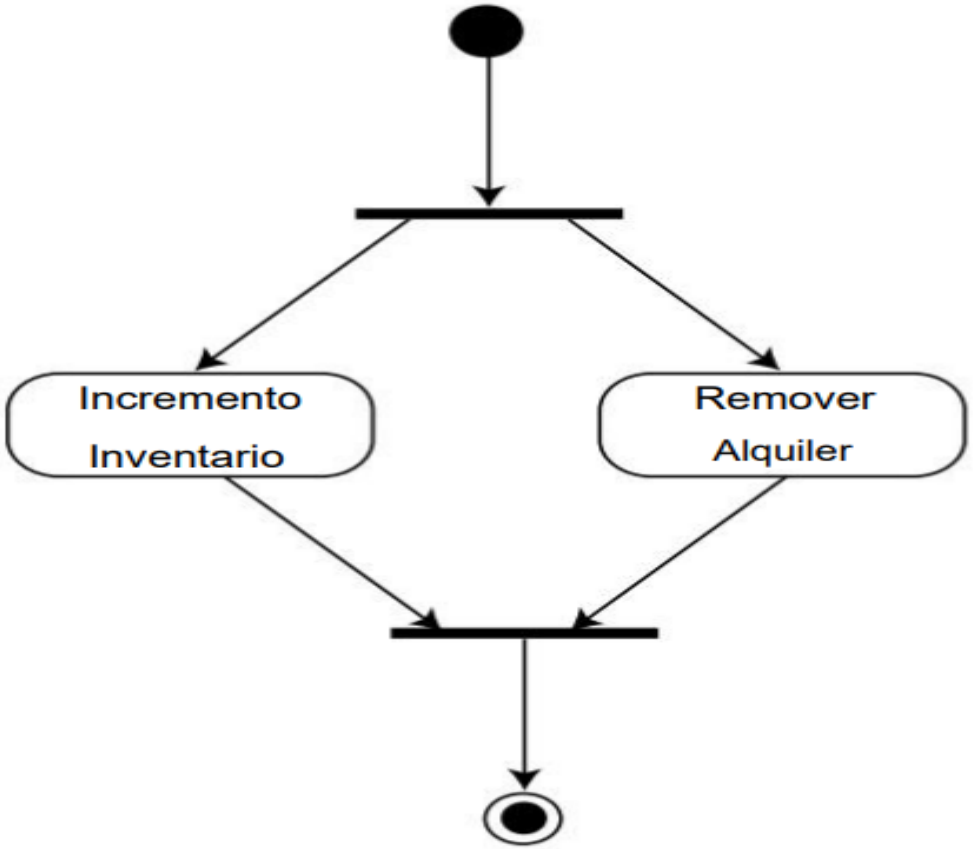
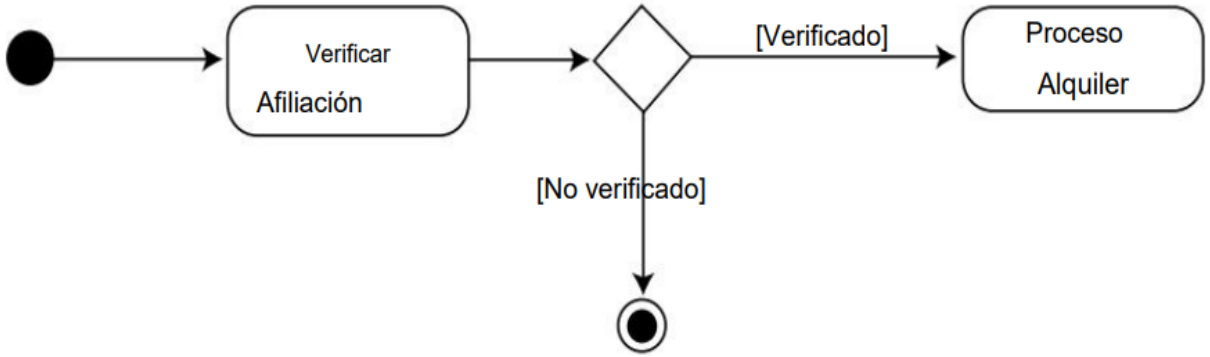


Diagrama de actividad para el caso de uso alquiler de película

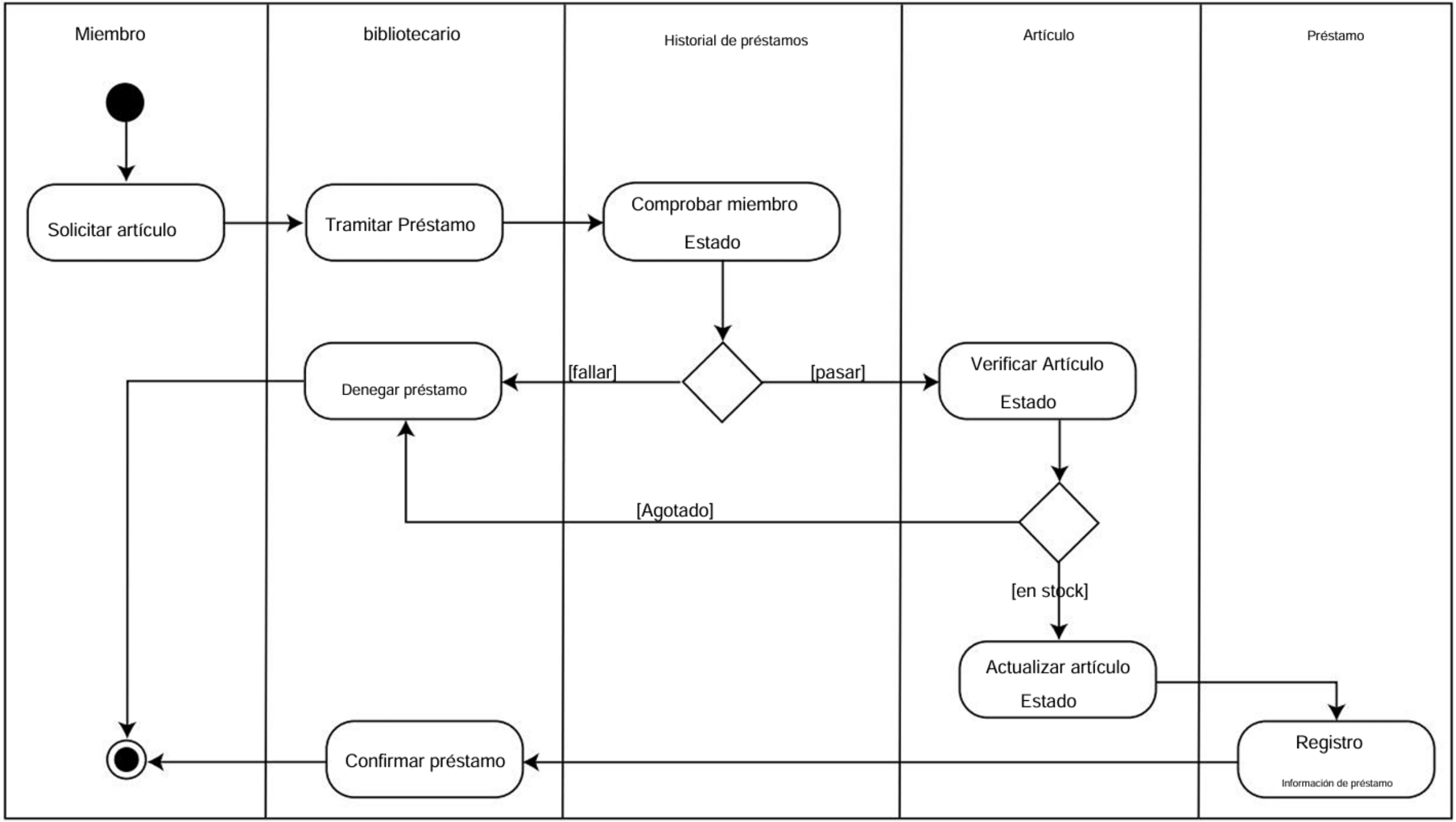
PROCESAMIENTO EN PARALELO / PUNTOS DE DECISION



Procesamiento paralelo representado en un diagrama de actividad



Indicación de puntos de decisión


Miembro
bibliotecario
Historial de préstamos
Artículo
Préstamo
Solicitar artículo
Tramitar Préstamo
Comprobar miembro
Estado
Denegar préstamo
[fallar]
[pasar]
Verificar Artículo
Estado
[Agotado]
[en stock]
Actualizar artículo
Estado
Confirmar préstamo
Registro
Información de préstamo

DISEÑO DE LA INTERFAZ GRAFICA DE USUARIO

10

DISEÑO DE LA GUI

- En los sistemas de software modernos, uno de los aspectos más importantes de una aplicación es qué tan bien interactúa con los usuarios.
- El diseño de la interfaz de usuario no debe hacerse al azar; más bien, debe planificarse junto con el diseño de la lógica empresarial. El éxito de la mayoría de las aplicaciones se juzga por la respuesta de los usuarios hacia la aplicación.

DISEÑO DE LA GUI

- Si los usuarios no se sienten cómodos al interactuar con la aplicación y la aplicación no mejora la productividad del usuario, está condenada al fracaso.
- **Para el usuario, la interfaz es la aplicación.** No importa cuán prístino e inteligente pueda ser el código de lógica empresarial; si la interfaz de usuario está mal diseñada e implementada, la aplicación no será aceptable para los usuarios.

DISEÑO DE LA GUI

- El primer paso es realizar un análisis de tareas para descubrir cómo los usuarios interactuarán con el sistema.
- El análisis de tares se basa en: Caso de Uso, Escenario y Diagrama de actividad.
- Una vez que haya identificado y priorizado las tareas necesarias, puede desarrollar un boceto **prototipo** de las distintas pantallas que conformarán la interfaz de usuario.

PROTOTIPO DE UNA GUI

Un prototipo es un modelo visual preliminar que muestra cómo se verá y cómo funcionará la interfaz de usuario de una aplicación antes de ser completamente desarrollada. No es el producto final, pero guía su desarrollo y evita errores costosos.

POR QUE UN PROTOTIPO DE LA GUI?

- Diseño de interfaces de usuario de forma rápida.
- Validación de ideas con usuarios antes de invertir tiempo en desarrollo real.
- Comunicación entre diseñadores, desarrolladores y clientes.

POR QUE UN PROTOTIPO DE LA GUI?

- **Visualiza la idea**: Permite ver cómo lucirá el sistema antes de programarlo.
- **Facilita la retroalimentación**: Clientes y usuarios pueden probar el flujo y proponer mejoras.
- **Ahorra tiempo y dinero**: Detecta errores de diseño o navegación antes de invertir en desarrollo.
- **Aclara requerimientos**: Mejora la comunicación entre analistas, diseñadores y desarrolladores.
- **Mejora la experiencia de usuario (UX)**: Permite validar si la interfaz es fácil de usar.

PROTOTIPO DE UNA GUI

Puede incluir:

- **Pantallas simuladas** (inicio, formularios, menús, etc.)
- **Botones, cajas de texto y menús** posicionados como en la versión final
- **Navegación entre pantallas** para simular el flujo del usuario

Puede ser:

- **Estático** (solo imágenes del diseño)
- **Interactivo** (simula clics, navegación, transiciones, etc.)

UN PROTOTIPO PUEDE SER:

- **Básico o de baja fidelidad**: bocetos en papel o digitales con elementos simples (cuadros, botones, menús), sin funcionalidad real.
- **Interactivo o de alta fidelidad**: simulaciones digitales detalladas que permiten al usuario "navegar" entre pantallas y ver comportamientos esperados (sin lógica de negocio real).

EJEMPLO DE HERRAMIENTAS

Balsamiq

- **Especializada en prototipos de baja fidelidad.**
- Sus diseños intencionalmente parecen "dibujados a mano“.
- Ideal para bocetar ideas rápidamente y enfocarse en la estructura, no en el diseño visual.

Figma, **Adobe XD** y **Sketch**

Son herramientas **más avanzadas**, orientadas a prototipos de **media y alta fidelidad**

Aunque puedes crear prototipos simples con ellas, su enfoque es más visual, preciso y con mayor realismo

La creación de prototipos de alta fidelidad se centra en la experiencia del usuario (UX), la interfaz de usuario (UI), la interacción y la presentación visual del sistema antes de la codificación real.

POSIBLES PANTALLAS Y FUNCIONALIDADES INCLUIDAS:

- **Página de inicio**: ver vuelos (sin iniciar sesión)
- **Registro/Login de cliente**
- **Panel del cliente**: buscar vuelos, reservar, ver historial, cancelar
- **Asignación automática de millas y descuentos**

- **Panel agente de boletos**: ver/editar vuelos
- **Notificaciones por correo simulado**
- **Gestión de vuelos y asientos disponibles**
- **Clasificación de usuarios: corporativo/minorista**

POSIBLES PANTALLAS Y FUNCIONALIDADES INCLUIDAS:

Procesos clave: reserva de vuelo, registro, modificación de reserva, cancelación, ver estado de vuelo.

Pantallas: página de inicio, buscar vuelos, registro, selección de vuelo, reserva, consultar reservas,

Estilo visual: colores, tipografía, tomar en cuenta nombre, slogan y logo del grupo.

"Mini prototipo funcional" usando Visual Studio + ASP.NET (o Blazor) con pantallas dummy ligadas al prototipo para mostrar que luego podrá desarrollarse. Muestra cómo se vería la pantalla final, pero no ejecuta procesos ni está conectada a la base de datos. Visualizar, probar o presentar el flujo del sistema sin necesidad de tener el código o la lógica implementada todavía.

ALGUNAS IDEAS

Tipo de pantalla simulada	Descripción	Ejemplo
Prototipo visual	Pantalla creada en Figma, Axure, XD, etc., sin código.	Simula un flujo entre pantallas.
HTML estático	Página web hecha con HTML/CSS pero sin programación.	Se puede abrir en navegador, pero no guarda ni procesa datos.
Formulario vacío en Visual Studio	Formulario ASP.NET o WinForms con codigo sencillo.	Muestra la interfaz pero no realiza operaciones con base de datos.

RECORDAR

- Un solo caso puede generar múltiples escenarios. Por ejemplo, para el caso de uso alquiler de películas hay dos posibilidades:

A) Describir lo que sucede cuando una película está disponible.

B) Describir lo que sucede cuando una película no está disponible.

Los documentos de diseño deben tener referencias cruzadas y revisarse continuamente según sea necesario.

ERRORES COMUNES QUE SE DEBEN EVITAR EN DURANTE EL ANALISIS:

- Tratar de modelar la solución completa al mismo tiempo: al desarrollar sistemas complejos, divida el diseño y el desarrollo del sistema en componentes manejables. Plan para producir el software en fases. Esto proporcionará ciclos más rápidos de modelado, desarrollo, prueba y lanzamiento.
- Esforzarse por crear un modelo perfecto: ningún modelo será perfecto desde el principio. Los modeladores exitosos entienden que el proceso de modelado es iterativo y que los modelos se actualizan y revisan continuamente a lo largo del ciclo de desarrollo de la aplicación.